

**UNIVERSITY OF MINDANAO TAGUM BRANCH
BACHELOR OF SCIENCE IN COMPUTER SCIENCE**

Department of Computing Education

Visayan Village, Tagum City, Philippines



**Course Requirements in
CS Professional Elective 1 (CSE 7/L)**

FUEL SCOUT: NEAR GAS STATION FINDER

Submitted to:

BENJAMIN M. MAHINAY

Submitted by:

BISAS, ALFRED ANGELO A.

BUENA, GABRIEL JEE M.

CAMARILLO, JOHN NIKOLAI S.

SAM, JASPER DAVIE C.

MAY 21, 2026

BACKGROUND OF THE STUDY

According to Bruglieri et al., (2021), motorists face problems whenever the fueling stations become crowded or are poorly distributed on their route. According to the scholars, route optimization is vital in minimizing inefficiencies associated with long travels. The research underscores the importance of smart routing systems that can effectively identify optimal routes while taking into consideration the availability of the fuel stations. In addition, according to Schoenberg and Dressler (2021), inefficient routing leads to lengthy waiting times and inefficiencies in the transportation process. They proposed an adaptive routing approach that effectively evaluates travel routes and station availability. Their findings clearly indicate that smart routing systems greatly minimize travel inefficiencies.

As per the study and analysis of traffic conducted by TomTom Traffic Index (2025), Davao City is still considered one of the traffic congested cities in the Philippines with 66.2% congestion. The results showed that motorists spend around 168 hours each year as a result of traffic congestion; moreover, traveling 10 kilometers in the city takes about 34 minutes and 17 seconds during peak hours. Based on the above findings, it can be deduced that motorists face challenges when it comes to finding efficient routes due to traffic congestion and increased number of vehicles in the city. Therefore, route optimization and intelligent navigation tools should be employed to enable motorists find faster routes. Furthermore, Ferreras (2026) argued that Davao City is one of the traffic congested cities in the Philippines. As a result of this, motorists in the city encounter long distances and high fuel consumption. Due to this, motorists are faced with difficulties locating nearby fuel stations and efficient routes in the city.

SYSTEM OVERVIEW

In view of traffic congestion and difficulties in transportation, it will be difficult for motorists to locate nearby fuel stations as well as find out how to use efficient routes. From studies and traffic reports, it can be observed that inefficiency in route selection by motorists increases the amount of time spent traveling and increases fuel consumption. Therefore, in order to solve these problems, the Nearest Gas Station project is made, a location-based web application that helps users find out nearby fuel

stations and how to use efficient routes. Furthermore, the system will obtain gasoline station data from Overpass API and then apply the Haversine formula to calculate the proximity of gasoline stations from the user's current position. Merge sort and binary search algorithms are used to determine the nearest gas stations while OpenRouteService is used to calculate actual driving routes, driving distance, and driving time.

SYSTEM ARCHITECTURE

The system processes user location through the following pipeline:

- **User Location**
 - Fetches the user location.
- **Overpass API (Haversine algorithm + R-tree Internally)**
 - Returns the gas stations within 5km of Radius based on the user location.
- **Gas Filter (Binary Search + Haversine + merge-sort algorithm)**
 - Filters three nearest gas stations.
- **ORS API (Dijkstra algorithm)**
 - Calculates and returns shortest road route from user to the selected gas station.
- **Leaflet Map**
 - Displays user location, three nearest stations, and the selected route.

ALGORITHM SECTION

Haversine Algorithm

The Haversine algorithm calculates the great-circle distance between two GPS coordinates on Earth's surface. Unlike Euclidean distance, it accounts for Earth's curvature, making it accurate for real-world geographic calculations. In this project it measures the distance between the user's location and each gas station within the 5km radius.

R-tree Algorithm

Overpass API uses an R-Tree spatial index to efficiently locate gas stations within the 5km radius. Rather than scanning all OSM nodes, it traverses a hierarchical tree of geographic bounding boxes, pruning branches that fall outside the query area. This reduces the search space dramatically, returning only nodes relevant to the user's location.

Merge-sort Algorithm

Merge Sort is used twice in GasFilter: first to sort the gas station array by ID, and second to sort the distance array from nearest to farthest. Sorting by ID is specifically required to enable Binary Search without a sorted array, Binary Search cannot function. Merge Sort was chosen for its consistent $O(n \log n)$ time complexity.

Binary Search Algorithm

Finds a target value inside a sorted array by repeatedly checking the middle entry and cutting the search range in half. It matches each sorted distance entry back to its full gas station record using the station ID this lets the project recover station details efficiently after sorting.

Dijkstra Algorithm

Dijkstra's algorithm finds the shortest path between nodes in a weighted graph. Road networks are naturally represented as graphs where intersections are nodes and roads are weighted edges. ORS (OpenRouteService) uses Dijkstra internally to calculate the optimal road route from the user's location to the selected gas station.

The system successfully combines multiple algorithms into a cohesive pipeline to solve a real-world navigation problem. Haversine calculates accurate GPS-based distances, Merge Sort organizes the data for efficient lookup, and Binary Search retrieves full station records with minimal computation. Together with Overpass API's spatial querying and ORS's Dijkstra-powered road routing, the system delivers the three nearest gas stations and a navigable route to the user in a seamless and efficient manner.

RESULT

The result of the concurrent website utilizes 5 Algorithms used to navigate through the map and display nearby gas stations.

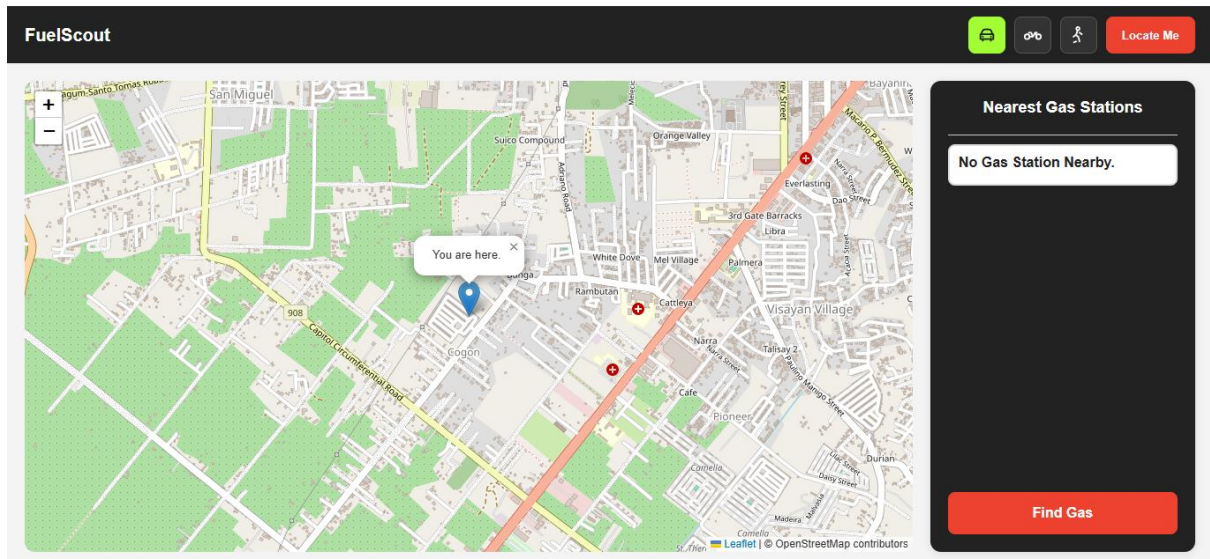


Figure 1

Geolocation API

The Geolocation API locates the current location of the user through its browser. As shown in Figure 1, the user's location was retrieved by the system and displayed on the map.

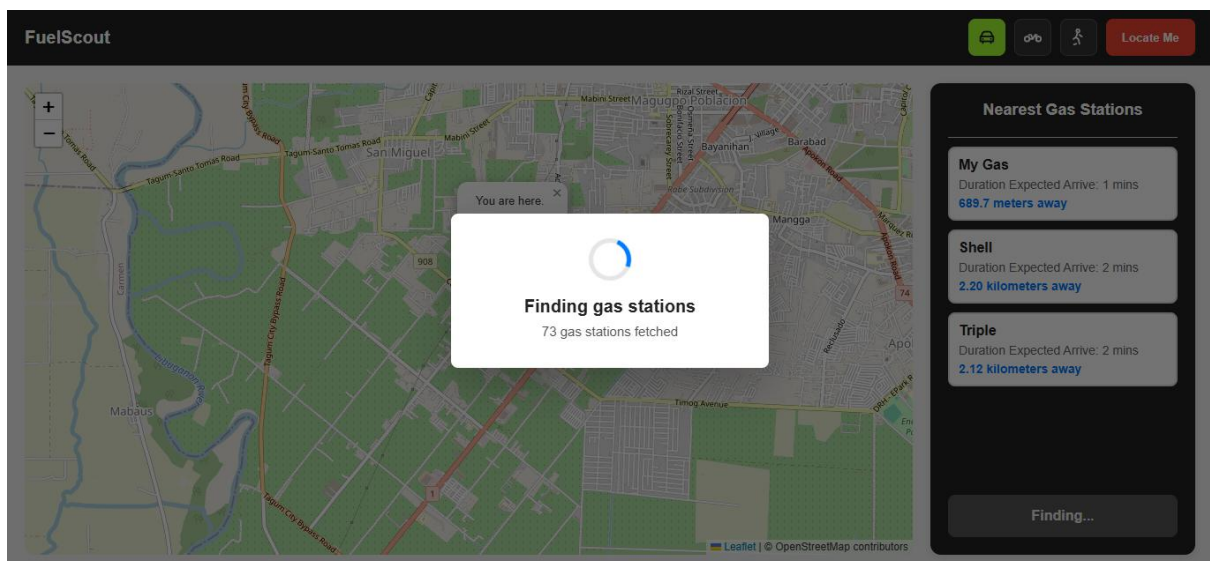


Figure 2.1

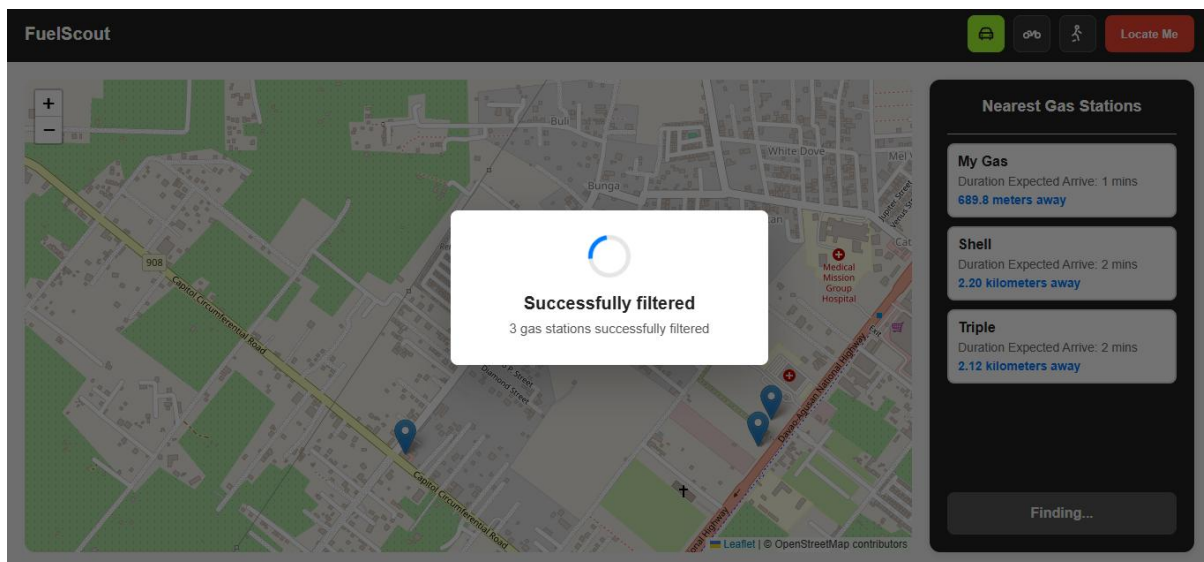


Figure 2. 2

Overpass API

OpenStreetMap data is utilized through the Overpass API to find nearby gas stations. When the user clicks “Find Gas,” it displays a loading screen shown in figure 2.1 and 2.2 and sends the user’s latitude and longitude to the Overpass API. Overpass then searches for fuel stations within a 5000-meter radius using a spatial indexing system to efficiently locate nearby map objects. After the gas stations are retrieved, the Haversine formula calculates the straight-line distance between the user’s location and each gas station based on their latitude and longitude. Merge sort is then used to arrange the gas stations from nearest to farthest. Finally, binary search matches each sorted distance result back to its complete gas station object, allowing the app to display the full station details.

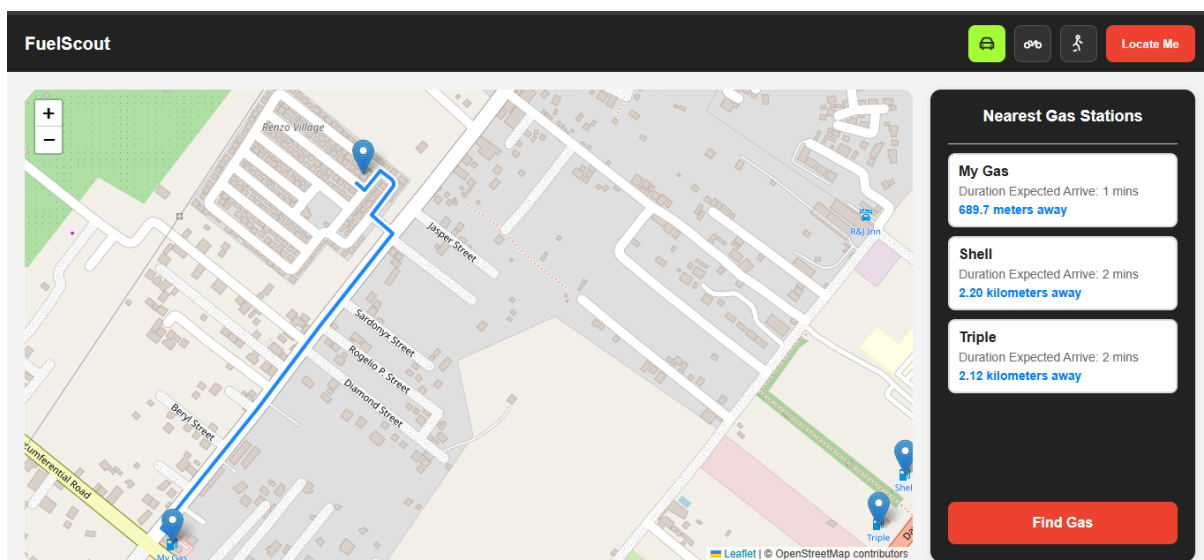


Figure 3

OpenRouteService API

The road route distance between the user and a chosen gas station is determined by OpenRouteService. As seen in Figure 3, it generates the optimal path from the user's location to the gas station using shortest path algorithms. The program uses OpenRouteService to draw the path when the user clicks on a gas station card. Additionally, it offers a variety of transportation options, including walking, cycling, and driving. The route computation is altered when a travel mode is chosen, which could have an impact on the projected journey time and distance.

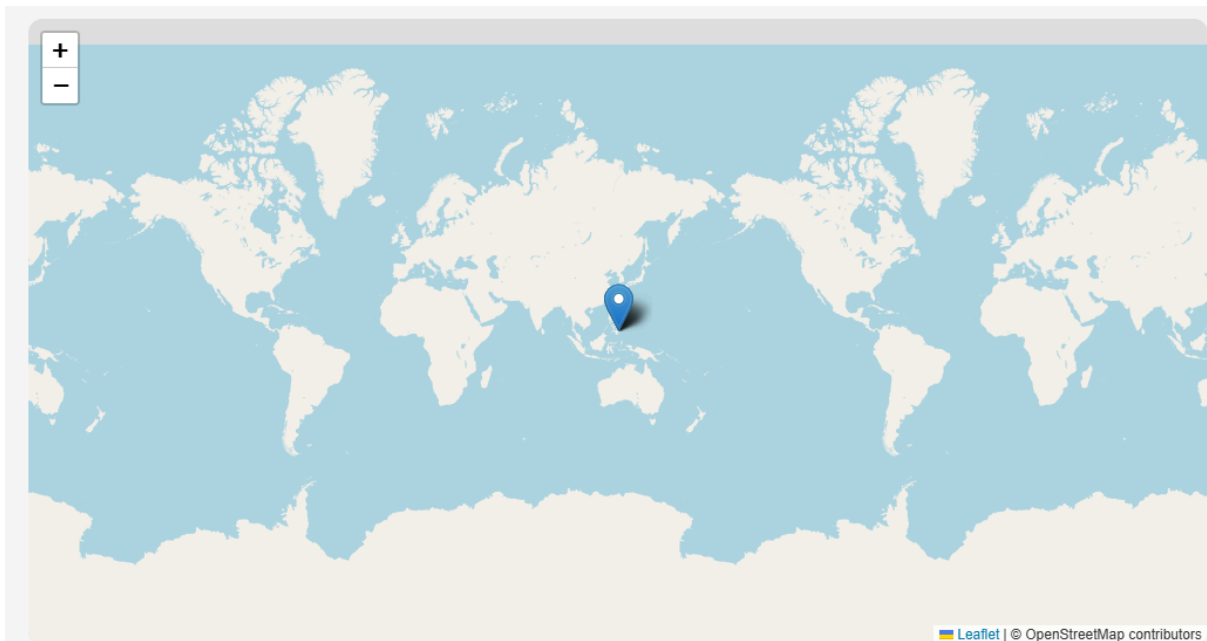


Figure 4

Leaflet and OpenStreetMap Tile API

The interactive map interface was made with Leaflet, while the website's map images came from OpenStreetMap. As shown in Figure 4, Users could interact with the location of neighboring gas stations and create routes using the map.

SCOPE AND LIMITATIONS

This system is a web-based application designed for users who need a quick map view of nearby gas stations and shortest available routes to reach a desired gas station. Its main purpose is to help users in locating the nearest gas station based on their current location and the available station data and routing information. This

system aims to provide convenience, accessibility, and route guidance for users who are looking for gas stations within the supported area.

The “Nearby Gas Station Finder” application is specifically designed for gas station searching only. It does not include other types of businesses such as restaurants, convenience stores, repair shops, or commercial establishments. It also does not include the current gas prices for that specific gas station. The system is centered on filtering and displaying the most nearby gas stations, along with route information that helps users identify a shorter path from a given location.

Scope of the System

- The system is web-based and can be accessed through a browser.
- It is designed for users who need to locate nearby gas stations within 5 kilometers radius.
- It focuses on finding the nearest gas station based on available location data.
- It provides route guidance using integrated mapping and routing services.
- It displays gas station information within the supported system data (Name, distance, and estimated time duration of arrival)
- It allows users to choose whether they are in a car, bike, or traveling on foot, as this affects the estimated time duration of arrival.
- It helps users to identify short routes toward selected gas stations.
- The system is intended only for gas station-related searching and navigation.

Limitations of the System

- The system cannot filter gas stations by specific gas station name.
- The system can only filter up to 3 gas stations.
- The location and routing data are dependent on the available map and API data.
- The system cannot automatically fetch newly registered gas stations unless the data source is updated.

- It is solely based on the user's proximity and cannot search for different areas.
- Newly constructed roads may not be tracked immediately if they are not yet available in OpenRouteService or the mapping API.
- The system does not include businesses other than gas stations.
- Gas prices are not included as information.
- The routing feature may be affected by API availability, internet connection, or service limitations.
- The accuracy of the results depends on third party APIs and the correctness of the stored gas station data.

REFERENCES

- Bruglieri, M., Mancini, S., & Pisacane, O. (2021). A more efficient cutting planes approach for the green vehicle routing problem with capacitated alternative fuel stations. *Optimization Letters*, 15(8), 2813–2829. <https://doi.org/10.1007/s11590-021-01714-3>
- TomTom Traffic Index. (2025). Davao City traffic report. TomTom. <https://www.tomtom.com/traffic-index/davao-city-traffic/>
- Ferreras, V. A. (2026, January 21). Traffic index: PH most congested in Asia, Davao City still most congested in PH. GMA News Online. <https://www.gmanetwork.com/news/topstories/nation/973708/ph-most-congested-in-asia-davao-city-still-most-congested-in-ph/story/>
- Schoenberg, S., & Dressler, F. (2021). Reducing waiting times at charging stations with adaptive electric vehicle route planning. arXiv. <https://arxiv.org/abs/2102.06503>